МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

# Отчет по лабораторной работе №3

Специальность АС-68/24

Выполнил
М. В. Красько,
студент группы АС-68/24

Проверил
А.А. Крощенко,
Доцент Кафедры ИИТ,
«  » 2026 г.

Брест 2026

**Цель:** на практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

**Вариант 5**

- Mushroom Classification
- Определить, является ли гриб ядовитым или съедобным
- **Задания:**
  1. Загрузите данные и преобразуйте все категориальные признаки в числовые (например, с помощью One-Hot Encoding);
  2. Разделите данные на обучающую и тестовую части;
  3. Обучите классификаторы k-NN, Decision Tree и SVM;
  4. Рассчитайте точность и полноту (precision и recall) для класса "ядовитый";
  5. Сделайте вывод о том, какой классификатор лучше всего справляется с этой задачей, где цена ошибки очень высока.

Код программы:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import precision_score, recall_score, accuracy_score

# загрузка данных
df = pd.read_csv("mushrooms.csv")
df = df.replace("?", pd.NA).dropna()

# признаки
y = df["class"]
X = pd.get_dummies(df.drop(columns="class"))

# разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print("Набор данных Mushroom")
print("Количество объектов:", X.shape[0])
print("Количество признаков:", X.shape[1])
print("Обучающая выборка:", X_train.shape[0])
print("Тестовая выборка:", X_test.shape[0])

# модели
models = {
    "k-NN": make_pipeline(StandardScaler(), KNeighborsClassifier()),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": make_pipeline(StandardScaler(), SVC()),
}

results = {}

print("\n==================== РЕЗУЛЬТАТЫ МОДЕЛЕЙ ====================")
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
```

```
            precision_p = precision_score(y_test, y_pred, pos_label="p",
zero_division=0)
            recall_p    = recall_score(y_test, y_pred, pos_label="p",
zero_division=0)
          accuracy    = accuracy_score(y_test, y_pred)

          print(f"\nМодель: {name}")
          print(f"  Точность для 'p' (Precision): {precision_p:.4f}")
          print(f"  Полнота для 'p' (Recall):     {recall_p:.4f}")
          print(f"  Общая точность (Accuracy):    {accuracy:.4f}")

          results[name] = recall_p

      best_model = max(results, key=results.get)
      worst_model = min(results, key=results.get)

      print("\n========== ВЫВОД ==========")
      print(f"Лучшая модель: {best_model}")
      print(f"Полнота (p) = {results[best_model]:.4f}, "
            f"худшая — {worst_model} "
            f"({results[worst_model]:.4f}).")
```

**Результат:**

Набор данных Mushroom
Количество объектов: 5644
Количество признаков: 98
Обучающая выборка: 3950
Тестовая выборка: 1694

========== РЕЗУЛЬТАТЫ МОДЕЛЕЙ ==========

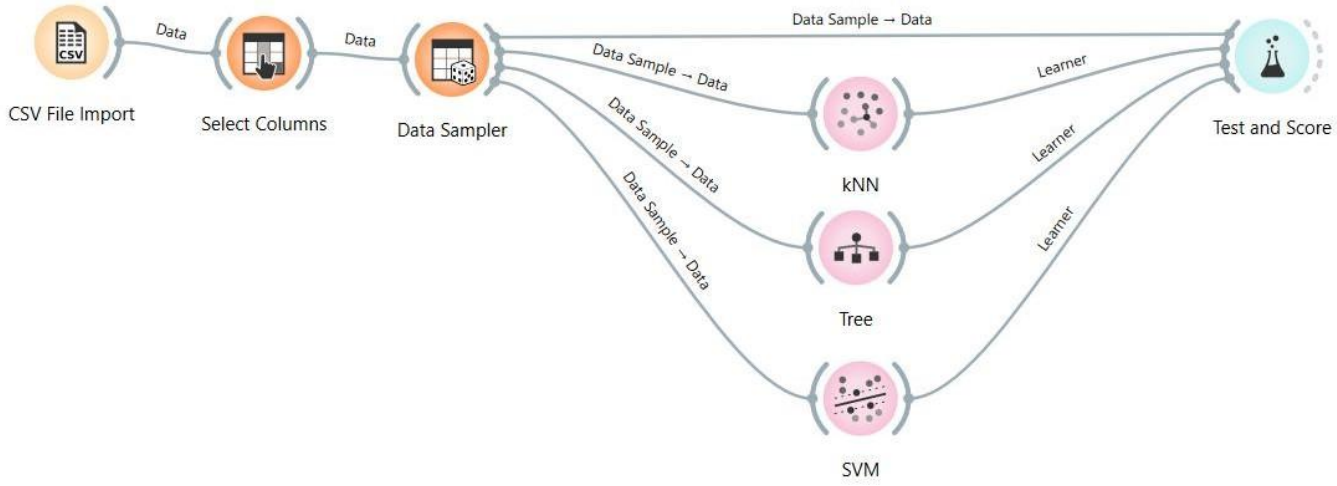
Модель: k-NN
  Точность для 'p' (Precision): 1.0000
  Полнота для 'p' (Recall):     0.9969
  Общая точность (Accuracy):    0.9988

Модель: Decision Tree
  Точность для 'p' (Precision): 1.0000
  Полнота для 'p' (Recall):     1.0000
  Общая точность (Accuracy):    1.0000

Модель: SVM
  Точность для 'p' (Precision): 1.0000
  Полнота для 'p' (Recall):     1.0000
  Общая точность (Accuracy):    1.0000

========== ВЫВОД ==========
Лучшая модель: Decision Tree
Полнота (p) = 1.0000, худшая — k-NN (0.9969).

**ORANGE**




Evaluation results for target p

| Model | AUC | CA | F1 | Prec | Recall | MCC |
|---|---|---|---|---|---|---|
| kNN | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 | 1.000 |
| Tree | 0.984 | 0.985 | 0.984 | 1.000 | 0.969 | 0.970 |
| SVM | 1.000 | 0.999 | 0.999 | 1.000 | 0.998 | 0.998 |

**Вывод:** на практике сравнила работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научилась подбирать гиперпараметры моделей и оценивать их влияние на результат.